

SINITT G2

Informe Técnico de Control de Calidad y Carga de Datos

Equipo G2 · Grow Data · Medellín, Mayo 2026

INFORME TÉCNICO DE CONTROL DE CALIDAD Y CARGA DE DATOS EN SUPABASE

Autor: Angy Xiomara Arango Ortiz – Mánager de Proyecto

Proyecto: Análisis de Movilidad e Infraestructura de Tráfico - Medellín

Fase: Carga, Homologación y Validation de Base de Datos

1. Introducción y Estado de la Carga

En mi rol como mánager del proyecto, coordiné y ejecuté la migración final de los datos hacia el servidor de Supabase (PostgreSQL), consolidando un universo total de 1.393.059 registros puros.

Tras aplicar los scripts de auditoría interna directamente en el motor de la base de datos, certifiqué que el porcentaje de duplicados de fila completa es del 0% en todas las tablas, garantizando una infraestructura limpia, robusta y optimizada para los análisis en Python y el despliegue del dashboard interactivo en Lovable.

2. Diagnóstico, Tratamiento y Diccionario Técnico por Tabla



TABLA: incidentes_viales

Total Registros Logrados: 270.693

Número de Columnas Cargadas: 13

Tratamiento y Decisiones de Carga: Al revisar el archivo original en la etapa previa a la carga, el dataset parecía estancarse en cerca de 250.000 registros debido a la gran cantidad de celdas vacías de origen. El planteamiento inicial del equipo sugería la eliminación masiva de las filas incompletas; sin embargo, identifiqué que esto nos costaría la pérdida crítica de más de 20.000 datos históricos de accidentalidad. Para salvar esta información, tomé la decisión estratégica de no eliminar ningún

registro por campos vacíos, sino aplicar una técnica de homologación en Python (Google Colab): cada celda de texto sin información fue rescatada e imputada bajo la etiqueta estándar de "NONE". Corriendo auditorías directas en el SQL Editor de Supabase, se validó el impacto real de mi decisión, logrando identificar el volumen exacto de datos que fueron protegidos de la depuración destructiva:

Estructura y Tipos de Datos en el Servidor:

id (bigint): Llave Primaria. Entero largo indexado.

fecha_accidentes (timestamp without time zone): Fecha y hora exacta registrada para el siniestro.

año / mes / numcomuna (integer): Desgloses numéricos creados para acelerar filtros temporales y de zonas.

clase_accidente / gravedad_accidente / comuna / barrio / diseño (text): Cadenas de caracteres categóricas. (Campos tratados con "NONE" en celdas vacías).

latitud / longitud (double precision): Coordenadas numéricas de precisión decimal flotante.

location (jsonb): Objeto JSON binario seguro {} para compatibilidad con mapas geográficos (GIS).



TABLA: paradas

Total Registros Logrados: 10.043

Número de Columnas Cargadas: 14

Tratamiento y Decisiones de Carga: Esta tabla representó el reto técnico más crítico durante el proceso de carga. Juan informó inicialmente que no se trabajaría con la columna objectid (Identificador de Objeto) y que debía omitirse. Al realizar las pruebas de carga en el servidor, descubrí que si eliminábamos esa columna, el motor de Supabase interpretaba erróneamente múltiples filas de rutas como duplicados espaciales, lo que encogía la base de datos catastróficamente a tan solo 3.590 registros. Evaluando el impacto, coloqué el criterio de mantener obligatoriamente esta columna y fijarla como la Llave Primaria oficial de la tabla, puesto que audité que este Identificador Único era el único campo nativo donde los datos no se repetían bajo ninguna circunstancia. Con esta decisión, se salvaron 6.453 registros de infraestructura, consolidando el inventario real y completo de 10.043 paradas de transporte de Medellín.

Estructura y Tipos de Datos en el Servidor:

objectid (integer): Llave Primaria. Identificador de Objeto entero que rescató el volumen de la tabla.

id_paradero / id_parada / id_ruta / nro_parada (integer): Identificadores numéricos del sistema de control de transporte.

nombre_ruta / codigo_ruta / tipo_ruta / empresa / recorrido / sistema_ruta / tipo_parada (text): Variables de texto descriptivas de las operaciones de buses y cuencas.

longitud / latitud (double precision): Coordenadas decimales exactas de ubicación espacial.

TABLA: velocidad_trafico

Total Registros Logrados: 681.302

Número de Columnas Cargadas: 19

Tratamiento de Carga: Al ser el volumen más denso del proyecto, se structured de manera indexada usando la columna id como texto para agilizar el consumo de datos. Se ejecutaron limpiezas automáticas para descartar lecturas corruptas causadas por desfases lógicos de tiempo en los sensores de la ciudad.

Estructura y Tipos de Datos en el Servidor:

id (text): Llave Primaria. Cadena de caracteres identificadora de registro.

fecha_hora (timestamp without time zone): Registro de fecha y hora nativo.

fecha (date): Formato de fecha limpia (YYYY-MM-DD).

tv_s / corredor / dia / mes / año / hora / diasem (integer): Variables analíticas de tiempo en segundos, códigos de tramos y desgloses de calendario.

nombre_corredor / flujo_tipo / sentido / inicio / fin / nombre_dia (text): Cadenas de texto operativas de la malla vial.

longitud_km / tv_minutos / velocidad_km_h / tv_min_km (double precision): Métricas decimales para el cálculo de velocidades comerciales.

TABLA: intensidad

Total Registros Logrados: 431.490

Número de Columnas Cargadas: 21

Tratamiento de Carga: Durante la carga en Supabase, se limpiaron los "registros espejo" duplicados, los cuales eran generados por sensores bidireccionales que duplicaban la volumetría de tráfico en tramos compartidos.

Estructura y Tipos de Datos en el Servidor:

id (bigint): Llave Primaria. Soporte numérico para alta densidad de registros.

fecha_trafico (timestamp with time zone): Fecha y hora parametrizada con la zona horaria de Colombia.

fecha (date): Fecha limpia para análisis cronológicos rápidos.

hora / dia_num / mes_num / ano / intensidad / categoria_1 / categoria_3 / ocupacion (integer) y categoria_2 (bigint): Contadores numéricos de volumen vehicular desglosado por categorías de transporte.

dia / corredor / sentido / operacion / tipo_sistema / comuna / nombre_comuna / mes (text): Datos geográficos y clasificadores de texto.

velocidad_km_h / longitud / latitud (double precision): Atributos decimales del comportamiento del flujo y su georreferenciación.



TABLA: rutas

Total Registros Logrados: 248

Número de Columnas Cargadas: 9

Tratamiento de Carga: Se consolidaron trazos limpios unificando códigos duplicados que se generaban originalmente por ligeras variaciones de trayecto entre las rutas de ida y vuelta.

Estructura y Tipos de Datos en el Servidor:

id_ruta (integer): Llave Primaria. Identificador numérico del trazo.

fecha (timestamp without time zone): Registro de actualización temporal.

nombre / codigo / sistema / tipo / empresa / recorrido (text): Textos de control de operadores y líneas de servicio público (Metro, buses, colectivos).

longitud (double precision): Longitud decimal del trayecto en kilómetros.



TABLA: camaras

Total Registros Logrados: 73

Número de Columnas Cargadas: 7

Tratamiento de Carga: Se realizó una validación manual registro por registro. Mantuve las columnas que contienen los enlaces de verificación de infraestructura urbana y el objectid (Identificador de Objeto) para asegurar el correcto cruce espacial con los incidentes.

Estructura y Tipos de Datos en el Servidor:

objectid (integer): Llave Primaria. Identificador de Objeto electrónico de la cámara de fotodetección.

direccion / localizacion / link_foto / link_detalle (text): Cadenas de texto con ubicaciones físicas y enlaces web de gestión.

longitud / latitud (double precision): Coordenadas geográficas decimales exactas para el mapeo espacial.

3. Soporte Tecnológico y Metodología de Validación

Para llevar a cabo con éxito todo el proceso de auditoría, control de calidad y validación estructural de estas seis bases de datos, utilicé un ecosistema avanzado de herramientas como soporte técnico de ingeniería:

Inteligencia Artificial: Implementada como copiloto estratégico para optimizar la lógica, acelerar la detección de anomalías y estructurar los reportes ejecutivos.

Python en Google Colab: Entorno de desarrollo utilizado para ejecutar scripts avanzados de limpieza, manipulación de grandes volúmenes de datos mediante diccionarios y análisis exploratorio previo (origen de las marcas de control "NONE").

Códigos Nativos en Supabase: Creación y ejecución de consultas SQL directas en el editor del servidor para interrogar el esquema global (information_schema), medir duplicados exactos por fila completa y verificar la asignación correcta de las llaves primarias.

4. Conclusión del Proceso

Asegurar técnicamente las bases de datos mediante la aplicación de tipos nativos de alta precisión (como double precision, bigint y jsonb) y defender el volumen de datos en incidentes_viales (rescatando con éxito los registros "NONE") y en paradas (salvando el objectid) nos entrega un ecosistema robusto, confiable y blindado.

Con esta carga finalizada en Supabase bajo mi supervisión, el equipo integrado por María Alejandra, Carol, Juan y Luz dispone de vía libre para avanzar con el modelado avanzado y el desarrollo de la aplicación web interactiva en Lovable sin riesgos de inconsistencias.